SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of india)
Re-Accredited by NAAC with 'A++' Grade
॥वसुधैव कुटुम्बकम्॥

CSE - Academic Year 2026-27
Java Programming Lab | Batch 2025-29

ASSIGNMENT : 25

Name of Student	Anushree Shinde
PRN No.	25070122041
Batch	CSE A - 2
Class	Java Programming Lab
Academic Year & Semester	2025-29 : Semester 3
Date of Submission	

TITLE : Write a Java program using the Thread class to create multiple threads and demonstrate their lifecycle.

Learning Objectives : To create and manage multiple threads using the Thread class.

Theory :

A thread is the smallest unit of execution within a program. Multithreading allows multiple tasks to run concurrently. A class can create a thread by extending Thread and overriding run(). The start() method starts execution; sleep() pauses a thread, and join() waits for a thread to finish. Thread states include New, Runnable, Waiting/Blocked, and Terminated.

Exercise :

1. Create a program where multiple threads print numbers simultaneously.

```java
class NumberThread extends Thread {
    String name;

    NumberThread(String name) {
        this.name = name;
    }

    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(name + ": " + i);

            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}

public class ThreadNumbers {
    public static void main(String[] args) {
        NumberThread t1 = new NumberThread("Thread 1");
        NumberThread t2 = new NumberThread("Thread 2");

        t1.start();
        t2.start();
    }
}
```

OUTPUT :

```
dg57e12.argfile ThreadNumbers
Thread 1: 3
Thread 2: 4
Thread 1: 4
Thread 1: 5
Thread 2: 5
PS C:\Users\softlink\Desktop\java code>
```

2. Simulate bank deposit and withdrawal operations using two threads.

```
class BankAccount {
    int balance = 1000;
    synchronized void deposit(int amount) {
        balance = balance + amount;
        System.out.println("Deposited: " + amount);
        System.out.println("Balance: " + balance);
    }
    synchronized void withdraw(int amount) {
        if (balance >= amount) {
            balance = balance - amount;
            System.out.println("Withdrawn: " + amount);
            System.out.println("Balance: " + balance);
        } else {
            System.out.println("Insufficient balance!");
        }
    }
}
class DepositThread extends Thread {
    BankAccount account;
    DepositThread(BankAccount account) {
        this.account = account;
    }
    public void run() {
        account.deposit(500);
    }
}
class WithdrawThread extends Thread {
    BankAccount account;

    WithdrawThread(BankAccount account) {
        this.account = account;
    }

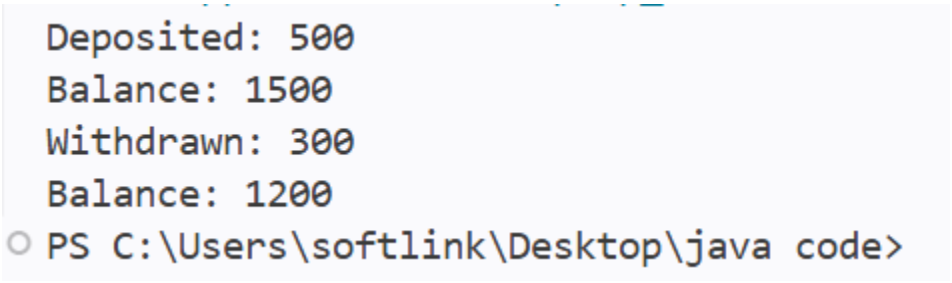
    public void run() {
        account.withdraw(300);
    }
}
```

```java
public class BankThreads {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();

        DepositThread t1 = new DepositThread(account);
        WithdrawThread t2 = new WithdrawThread(account);

        t1.start();
        t2.start();
    }
}
```

OUTPUT :



Outcome :

- Understood the concept of multithreading in Java.
- Learned to create multiple threads by extending the Thread class.
- Understood the use of run(), start(), sleep(), and join() methods.
- Learned about different thread lifecycle states, including New, Runnable, Waiting, and Terminated.
- Successfully implemented multithreaded programs for printing numbers and simulating bank deposit and withdrawal operations.

Conclusion :

This assignment demonstrated the use of the Thread class to create and manage multiple threads in Java. The programs illustrated concurrent number printing and bank transactions using separate threads. The assignment helped in understanding thread creation, lifecycle management, synchronization, and concurrent task execution.